

SmartMoldGuard 防霉管控系统 - DevOps 规划文档

编号: DDD-TAC-SmartMoldGuard-20251217-v1.0-DevOps

状态: Final

版本说明: 轻量级单服务器 DevOps 落地操作规划

依据: DDD-TAC-SmartMoldGuard-20251216-v1.0

目标: 实现成本效益高、资源消耗低的完整 DevOps 工具链

基于 SmartMoldGuard 防霉管控系统的 DDD 战术设计，以下是在单台服务器上实现轻量级 DevOps 工具链的详细操作规划。本规划兼顾成本效益、可维护性和扩展性，采用轻量级工具链降低资源消耗。

一、环境准备与架构设计

1.1 服务器资源配置建议

最低配置要求:

- CPU: 8核
- 内存: 32GB
- 存储: 500GB SSD
- 操作系统: Ubuntu 22.04 LTS

资源分配规划:

1	单台服务器 (32GB RAM, 8核)		
2			
3	Docker & Docker Compose (基础运行环境)		
4			
5			
6	DevOps工具区	服务运行区	数据存储区
7	(约6GB内存)	(约18GB内存)	(约8GB内存)
8			
9	GitLab CE	8个微服务	PostgreSQL
10	Jenkins	API网关	InfluxDB
11	Nexus		Redis
12	ELK Stack		
13	Prometheus		
14			
15			
16			

1.2 基础环境安装

1	# 1. 系统更新
2	sudo apt update && sudo apt upgrade -y
3	
4	# 2. 安装 Docker

```
5 curl -fSSL https://get.docker.com | sh
6 sudo usermod -aG docker $USER
7 newgrp docker
8
9 # 3. 安装 Docker Compose
10 sudo curl -L "https://github.com/docker/compose/releases/latest/download/docker-
11 compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose
12 sudo chmod +x /usr/local/bin/docker-compose
13
14 # 4. 配置 Docker 镜像加速（国内环境建议）
15 sudo mkdir -p /etc/docker
16 sudo tee /etc/docker/daemon.json <<EOF
17 {
18     "registry-mirrors": ["https://mirror.gcr.io", "https://docker-proxy.com"],
19     "log-driver": "json-file",
20     "log-opts": {"max-size": "10m", "max-file": "3"},
21     "memory": "28g",
22     "cpus": 7.5
23 }
24 EOF
25 sudo systemctl restart docker
26
27 # 5. 创建项目目录结构
28 mkdir -p
29 /opt/smartmoldguard/{devops,services,data/{postgres,influxdb,redis},logs,backups}
30 cd /opt/smartmoldguard
```

二、DevOps 工具链部署

2.1 Gitea (代码管理)

```
1 # 创建 Gitea 配置
2 cat > devops/docker-compose.gitea.yml <<EOF
3 version: '3.8'
4 services:
5     gitea:
6         image: gitea/gitea:1.21.5
7         container_name: gitea
8         restart: unless-stopped
9         environment:
10             - GITEA__database__DB_TYPE=postgres
11             - GITEA__database__HOST=postgres:5432
12             - GITEA__database__NAME=gitea_db
13             - GITEA__database__USER=gitea_user
14             - GITEA__database__PASSWD=gitea_password
15             - GITEA__server__DOMAIN=localhost
16             - GITEA__server__HTTP_PORT=8080
17             - GITEA__server__ROOT_URL=http://gitea.smartmoldguard.local:8080/
18             - GITEA__security__INSTALL_LOCK=true
19         ports:
20             - "8080:8080"
```

```

21     - "2224:22"
22     volumes:
23     - ./data/gitea:/data
24     - /etc/timezone:/etc/timezone:ro
25     - /etc/localtime:/etc/localtime:ro
26     networks:
27     - devops-net
28     depends_on:
29     - postgres
30     mem_limit: 1g
31     cpus: 0.5
32
33     postgres:
34     image: postgres:15-alpine
35     container_name: gitea-postgres
36     environment:
37     POSTGRES_DB: gitea_db
38     POSTGRES_USER: gitea_user
39     POSTGRES_PASSWORD: gitea_password
40     volumes:
41     - ./data/postgres/gitea:/var/lib/postgresql/data
42     networks:
43     - devops-net
44     restart: unless-stopped
45     mem_limit: 512m
46
47     networks:
48     devops-net:
49     driver: bridge
50 EOF
51
52 # 启动 Gitea
53 docker-compose -f devops/docker-compose.gitea.yml up -d
54
55 # 访问 http://<服务器IP>:8080，首次访问需要完成初始化配置
56 # 默认管理员账号: admin / admin123

```

Gitea 配置任务清单:

- ☐ 登录后修改管理员密码
- ☐ 创建 smartmoldguard 组织
- ☐ 创建 5 个微服务项目仓库 (对应 DDD 设计的微服务)
- ☐ 创建 api-gateway 仓库
- ☐ 创建 infra 仓库 (存放部署配置)
- ☐ 创建 shared-lib 仓库 (共享库)
- ☐ 配置 Deploy Keys 和 Webhooks

2.2 Woodpecker CI (CI/CD 流水线)

```
1  # 创建 woodpecker CI 配置
2  cat > devops/docker-compose.woodpecker.yml <<EOF
3  version: '3.8'
4  services:
5    woodpecker-server:
6      image: woodpeckerci/woodpecker-server:latest
7      container_name: woodpecker-server
8      restart: unless-stopped
9      environment:
10       - WOODPECKER_OPEN=false
11       - WOODPECKER_HOST=http://woodpecker.smartmoldguard.local:8081
12       - WOODPECKER_GITEA=true
13       - WOODPECKER_GITEA_URL=http://gitea:8080
14       - WOODPECKER_GITEA_CLIENT=34320244-9711-47cb-b123-565408b3f010
15       - WOODPECKER_GITEA_SECRET=your-gitea-secret
16       - WOODPECKER_AGENT_SECRET=your-agent-secret
17      ports:
18       - "8081:8000"
19      volumes:
20       - ./data/woodpecker/server:/var/lib/woodpecker
21      networks:
22       - devops-net
23      depends_on:
24       - gitea
25      mem_limit: 1g
26      cpus: 0.5
27
28    woodpecker-agent:
29      image: woodpeckerci/woodpecker-agent:latest
30      container_name: woodpecker-agent
31      restart: unless-stopped
32      command: agent
33      environment:
34       - WOODPECKER_SERVER=woodpecker-server:9000
35       - WOODPECKER_AGENT_SECRET=your-agent-secret
36       - WOODPECKER_HOSTNAME=agent-1
37       - WOODPECKER_MAX_WORKFLOWS=5
38      volumes:
39       - /var/run/docker.sock:/var/run/docker.sock
40      networks:
41       - devops-net
42      depends_on:
43       - woodpecker-server
44      mem_limit: 512m
45      cpus: 0.3
46
47  networks:
48    devops-net:
49      external: true
50  EOF
```

```
51
52 # 启动 woodpecker CI
53 docker-compose -f devops/docker-compose.woodpecker.yml up -d
54
55 # 访问 http://<服务器IP>:8081, 使用 Gitea 账号登录
```

Woodpecker CI 配置任务清单:

- ☐ 在 Gitea 中创建 OAuth 应用, 获取 Client ID 和 Secret
- ☐ 更新 Woodpecker CI 配置文件, 填入正确的 Gitea Client ID 和 Secret
- ☐ 访问 http://<服务器IP>:8081, 使用 Gitea 账号登录
- ☐ 为每个微服务仓库配置 CI/CD 流水线
- ☐ 创建 .woodpecker.yml 模板 (见下文)

2.3 制品库管理 (轻量级方案)

采用公共制品库 + 本地缓存的轻量级方案, 无需部署私有仓库, 大幅降低资源消耗:

- **Docker 镜像:** 使用 Docker Hub 公共仓库 + 本地 Docker 镜像缓存
- **Maven 依赖:** 使用 Maven Central + 本地 Maven 缓存
- **其他依赖:** 根据需要使用对应的公共仓库

配置任务清单:

- ☐ 在 Gitea 中配置 Docker Hub 凭据
- ☐ 配置 Maven 本地缓存
- ☐ 配置 Docker 镜像加速 (国内环境建议)

```
1 # 配置 Docker 镜像加速 (国内环境建议)
2 sudo tee /etc/docker/daemon.json <<EOF
3 {
4     "registry-mirrors": ["https://registry.docker-cn.com",
5     "https://docker.mirrors.ustc.edu.cn"],
6     "log-driver": "json-file",
7     "log-opts": {"max-size": "10m", "max-file": "3"}
8 }
9 EOF
10 sudo systemctl restart docker
```

2.4 Loki + Promtail (日志收集)

```
1 # 创建 Loki + Promtail 配置
2 cat > devops/docker-compose.loki.yml <<EOF
3 version: '3.8'
4 services:
5     loki:
6         image: grafana/loki:3.0.0
7         container_name: loki
8         restart: unless-stopped
```

```
9     ports:
10         - "3100:3100"
11     command: -config.file=/etc/loki/local-config.yaml
12     volumes:
13         - ./data/loki:/loki
14     networks:
15         - devops-net
16     mem_limit: 1g
17     cpus: 0.5
18
19     promtail:
20         image: grafana/promtail:3.0.0
21         container_name: promtail
22         restart: unless-stopped
23         command: -config.file=/etc/promtail/config.yml
24         volumes:
25             - ./devops/promtail/config.yml:/etc/promtail/config.yml
26             - /var/lib/docker/containers:/var/lib/docker/containers:ro
27             - /var/run/docker.sock:/var/run/docker.sock:ro
28         networks:
29             - devops-net
30         depends_on:
31             - loki
32         mem_limit: 512m
33         cpus: 0.3
34
35     grafana:
36         image: grafana/grafana:latest
37         container_name: grafana
38         restart: unless-stopped
39         ports:
40             - "3000:3000"
41         environment:
42             - GF_SECURITY_ADMIN_PASSWORD=admin123
43         volumes:
44             - ./data/grafana:/var/lib/grafana
45         networks:
46             - devops-net
47         depends_on:
48             - loki
49         mem_limit: 1g
50         cpus: 0.5
51
52     networks:
53         devops-net:
54             external: true
55 EOF
56
57 # 创建 Promtail 配置
58 mkdir -p devops/promtail
59 cat > devops/promtail/config.yml <<EOF
60 server:
61     http_listen_port: 9080
```

```
62     grpc_listen_port: 0
63
64 positions:
65     filename: /tmp/positions.yaml
66
67 clients:
68     - url: http://loki:3100/loki/api/v1/push
69
70 snippets:
71     pipeline_stages:
72         - json:
73             expressions:
74                 time: time
75                 level: level
76                 message: message
77                 service_name: service_name
78         - timestamp:
79             source: time
80             format: RFC3339Nano
81         - labels:
82             level:
83             service_name:
84
85 scrape_configs:
86     - job_name: system
87       static_configs:
88         - targets:
89             - localhost
90           labels:
91             job: varlogs
92             host: localhost
93             __path__: /var/log/*log
94
95     - job_name: docker
96       docker_sd_configs:
97         - host: unix:///var/run/docker.sock
98           refresh_interval: 5s
99       relabel_configs:
100     - source_labels: [__meta_docker_container_name]
101       regex: /(.*?)
102       target_label: container
103     - source_labels: [__meta_docker_container_log_stream]
104       target_label: log_stream
105     - source_labels: [__meta_docker_container_label_com_docker_compose_service]
106       target_label: service_name
107     pipeline_stages:
108         - json:
109             expressions:
110                 log: log
111                 stream: stream
112                 time: time
113         - timestamp:
114             source: time
```

```

115         format: RFC3339Nano
116     - labels:
117         stream:
118     - output:
119         source: log
120 EOF
121
122 # 启动 Loki + Promtail
123 docker-compose -f devops/docker-compose.loki.yml up -d
124
125 # 配置 Grafana 数据源
126 # 1. 访问 http://<服务器IP>:3000，登录后添加 Loki 数据源
127 # 2. URL: http://loki:3100
128 # 3. 保存并测试连接

```

2.5 Prometheus + Node Exporter + cAdvisor (监控告警)

```

1 # 创建 Prometheus 配置
2 mkdir -p devops/prometheus
3 cat > devops/prometheus/prometheus.yml <<EOF
4 global:
5     scrape_interval: 15s
6
7 scrape_configs:
8     - job_name: 'prometheus'
9       static_configs:
10         - targets: ['localhost:9090']
11
12     - job_name: 'node-exporter'
13       static_configs:
14         - targets: ['node-exporter:9100']
15
16     - job_name: 'cadvisor'
17       static_configs:
18         - targets: ['cadvisor:8080']
19
20     - job_name: 'eureka'
21       metrics_path: '/actuator/prometheus'
22       static_configs:
23         - targets: ['eureka-server:8761']
24
25     - job_name: 'spring-boot'
26       metrics_path: '/actuator/prometheus'
27       eureka_sd_configs:
28         - server: http://eureka-server:8761/eureka
29       relabel_configs:
30         - source_labels: [__meta_eureka_app_name]
31           target_label: application
32 EOF
33
34 # 创建监控 Docker Compose
35 cat > devops/docker-compose.monitoring.yml <<EOF

```

```
36 version: '3.8'
37 services:
38   prometheus:
39     image: prom/prometheus:latest
40     container_name: prometheus
41     restart: unless-stopped
42     ports:
43       - "9090:9090"
44     volumes:
45       - ./devops/prometheus/prometheus.yml:/etc/prometheus/prometheus.yml
46       - ./data/prometheus:/prometheus
47     command:
48       - '--config.file=/etc/prometheus/prometheus.yml'
49       - '--storage.tsdb.path=/prometheus'
50       - '--storage.tsdb.retention.time=30d'
51     networks:
52       - devops-net
53     mem_limit: 1g
54     cpus: 0.5
55
56   node-exporter:
57     image: prom/node-exporter:latest
58     container_name: node-exporter
59     restart: unless-stopped
60     ports:
61       - "9100:9100"
62     networks:
63       - devops-net
64     mem_limit: 256m
65     cpus: 0.2
66
67   cadvisor:
68     image: google/cadvisor:latest
69     container_name: cadvisor
70     restart: unless-stopped
71     ports:
72       - "8083:8080"
73     volumes:
74       - /:/rootfs:ro
75       - /var/run:/var/run:ro
76       - /sys:/sys:ro
77       - /var/lib/docker:/var/lib/docker:ro
78       - /dev/disk:/dev/disk:ro
79     networks:
80       - devops-net
81     mem_limit: 512m
82     cpus: 0.3
83
84 networks:
85   devops-net:
86     external: true
87 EOF
88
```

```
89 # 启动监控
90 docker-compose -f devops/docker-compose.monitoring.yml up -d
```

2.6 Eureka (服务注册与发现)

```
1 # 创建 Eureka 配置
2 cat > devops/docker-compose.eureka.yml <<EOF
3 version: '3.8'
4 services:
5   eureka-server:
6     image: springcloud/eureka-server:3.1.2
7     container_name: eureka-server
8     restart: unless-stopped
9     environment:
10      - EUREKA_SERVER_PORT=8761
11      - EUREKA_CLIENT_REGISTER_WITH_EUREKA=false
12      - EUREKA_CLIENT_FETCH_REGISTRY=false
13      - EUREKA_INSTANCE_HOSTNAME=eureka-server
14      - MANAGEMENT_METRICS_EXPORT_PROMETHEUS_ENABLED=true
15     ports:
16      - "8761:8761"
17     networks:
18      - devops-net
19     mem_limit: 512m
20     cpus: 0.3
21
22 networks:
23   devops-net:
24     external: true
25 EOF
26
27 # 启动 Eureka
28 docker-compose -f devops/docker-compose.eureka.yml up -d
```

2.7 Spring Cloud Config (配置中心)

```
1 # 创建 Spring Cloud Config 配置
2 cat > devops/docker-compose.config.yml <<EOF
3 version: '3.8'
4 services:
5   config-server:
6     image: springcloud/config-server:3.1.2
7     container_name: config-server
8     restart: unless-stopped
9     environment:
10      - SPRING_PROFILES_ACTIVE=native
11      - SPRING_CLOUD_CONFIG_SERVER_NATIVE_SEARCH_LOCATIONS=/config
12      -
13      SPRING_CLOUD_CONFIG_SERVER_GIT_URI=https://gitea.smartmoldguard.local/smartmoldguard/co
nfig-repo.git
14      - SPRING_CLOUD_CONFIG_SERVER_GIT_USERNAME=admin
```

```

14     - SPRING_CLOUD_CONFIG_SERVER_GIT_PASSWORD=admin123
15     - EUREKA_CLIENT_SERVICE_URL_DEFAULTZONE=http://eureka-server:8761/eureka
16     - MANAGEMENT_METRICS_EXPORT_PROMETHEUS_ENABLED=true
17     ports:
18     - "8888:8888"
19     volumes:
20     - ./data/config:/config
21     networks:
22     - devops-net
23     depends_on:
24     - eureka-server
25     mem_limit: 512m
26     cpus: 0.3
27
28     networks:
29     devops-net:
30     external: true
31 EOF
32
33 # 启动 Spring Cloud Config
34 docker-compose -f devops/docker-compose.config.yml up -d

```

访问地址:

- Gitea: http://<服务器IP>:8080
- Woodpecker CI: http://<服务器IP>:8081
- Grafana: http://<服务器IP>:3000
- Prometheus: http://<服务器IP>:9090
- Eureka: http://<服务器IP>:8761
- Spring Cloud Config: http://<服务器IP>:8888
- Loki: http://<服务器IP>:3100

三、DDD 微服务项目结构模板

3.1 Maven 父 POM 配置

```

1  <!-- 根目录 pom.xml -->
2  <project>
3      <modelVersion>4.0.0</modelVersion>
4      <groupId>com.smartmoldguard</groupId>
5      <artifactId>smartmoldguard-parent</artifactId>
6      <version>1.0.0-SNAPSHOT</version>
7      <packaging>pom</packaging>
8
9      <modules>
10         <module>device-service</module>
11         <module>control-service</module>
12         <module>ai-service</module>
13         <module>subscription-service</module>

```

```
14     <module>report-service</module>
15     <module>api-gateway</module>
16     <module>shared-library</module>
17 </modules>
18
19 <properties>
20     <java.version>17</java.version>
21     <spring-boot.version>3.2.0</spring-boot.version>
22     <spring-cloud.version>2023.0.0</spring-cloud.version>
23     <postgresql.version>42.6.0</postgresql.version>
24     <influxdb.version>6.10.0</influxdb.version>
25     <kafka.version>3.6.0</kafka.version>
26     <testcontainers.version>1.19.3</testcontainers.version>
27 </properties>
28
29 <dependencyManagement>
30     <dependencies>
31         <dependency>
32             <groupId>org.springframework.boot</groupId>
33             <artifactId>spring-boot-dependencies</artifactId>
34             <version>${spring-boot.version}</version>
35             <type>pom</type>
36             <scope>import</scope>
37         </dependency>
38         <dependency>
39             <groupId>org.springframework.cloud</groupId>
40             <artifactId>spring-cloud-dependencies</artifactId>
41             <version>${spring-cloud.version}</version>
42             <type>pom</type>
43             <scope>import</scope>
44         </dependency>
45     </dependencies>
46 </dependencyManagement>
47
48 <build>
49     <plugins>
50         <plugin>
51             <groupId>org.springframework.boot</groupId>
52             <artifactId>spring-boot-maven-plugin</artifactId>
53             <configuration>
54                 <image>
55                     <name>${project.artifactId}:${project.version}</name>
56                 </image>
57             </configuration>
58         </plugin>
59     </plugins>
60 </build>
61 </project>
```

3.2 单个微服务 DDD 结构

以 `device-service` 为例：

```
1  device-service/
2  |─ src/
3  |   |─ main/
4  |   |   |─ java/
5  |   |   |   |─ com/smartmoldguard/device/
6  |   |   |       |─ DeviceServiceApplication.java
7  |   |   |       |─ domain/                # 领域层
8  |   |   |       |   |─ model/              # 聚合根、实体、值对象
9  |   |   |       |   |   |─ Device.java
10 |   |   |       |   |   |─ Room.java
11 |   |   |       |   |   |─ ButtonMapping.java
12 |   |   |       |   |─ repository/          # 仓储接口
13 |   |   |       |   |   |─ DeviceRepository.java
14 |   |   |       |   |─ service/             # 领域服务
15 |   |   |       |       |─ DeviceManagementService.java
16 |   |   |       |─ application/             # 应用层
17 |   |   |       |   |─ dto/                 # DTO
18 |   |   |       |   |─ command/            # 命令
19 |   |   |       |   |─ query/              # 查询
20 |   |   |       |   |─ service/            # 应用服务
21 |   |   |       |       |─ DeviceAppService.java
22 |   |   |       |─ infrastructure/          # 基础设施层
23 |   |   |       |   |─ config/              # 配置
24 |   |   |       |   |─ persistence/         # 仓储实现
25 |   |   |       |       |─ DeviceRepositoryImpl.java
26 |   |   |       |   |─ messaging/           # 消息队列
27 |   |   |       |   |─ iot/                 # IoT 模拟客户端
28 |   |   |       |─ interfaces/              # 接口层
29 |   |   |       |   |─ rest/                 # REST API
30 |   |   |       |       |─ DeviceController.java
31 |   |   |       |   |─ event/                # 事件监听
32 |   |   |   |─ resources/
33 |   |   |       |─ application.yml
34 |   |   |       |─ logback-spring.xml
35 |   |   |─ test/
36 |   |       |─ unit/                          # 单元测试
37 |   |       |─ integration/                  # 集成测试
38 |   |       |─ resources/
39 |─ pom.xml
```

四、CI/CD 流水线实现

4.1 .woodpecker.yml 模板

```
1  # 在 Gitea 每个服务仓库根目录创建 .woodpecker.yml
2  pipeline:
3
4      检出代码:
5          image: woodpeckerci/plugin-git:latest
6          settings:
7              depth: 1
8
9      运行单元测试:
10         image: maven:3.9-eclipse-temurin-17
11         commands:
12             - mvn clean test
13         when:
14             branch: [main, develop]
15
16      代码质量检查:
17         image: maven:3.9-eclipse-temurin-17
18         commands:
19             - mvn checkstyle:check
20         when:
21             branch: [main, develop]
22
23      构建 Docker 镜像:
24         image: docker:24
25         commands:
26             - docker build -t ${CI_REPO_NAME}:${CI_COMMIT_SHA} .
27         volumes:
28             - /var/run/docker.sock:/var/run/docker.sock
29         when:
30             branch: [main, develop]
31
32      部署到测试环境:
33         image: docker:24
34         commands:
35             - docker-compose -f services/docker-compose.${CI_REPO_NAME}.yml pull
36             - docker-compose -f services/docker-compose.${CI_REPO_NAME}.yml up -d
37         volumes:
38             - /var/run/docker.sock:/var/run/docker.sock
39         when:
40             branch: develop
41
42      集成测试:
43         image: maven:3.9-eclipse-temurin-17
44         commands:
45             - mvn verify -Pintegration-test
46         when:
47             branch: [main, develop]
48
```

```

49 部署到生产环境:
50  image: docker:24
51  commands:
52    - docker-compose -f services/docker-compose.prod.yml pull ${CI_REPO_NAME}
53    - docker-compose -f services/docker-compose.prod.yml up -d ${CI_REPO_NAME}
54  volumes:
55    - /var/run/docker.sock:/var/run/docker.sock
56  when:
57    branch: main
58    event: push
59
60 发送通知:
61  image: plugins/slack
62  settings:
63    webhook:
64      https://hooks.slack.com/services/T00000000/B00000000/XXXXXXXXXXXXXXXXXXXXXXXXX
65    channel: '#ci-cd'
66    username: Woodpecker CI
67    message: "构建 ${CI_BUILD_STATUS}: ${CI_REPO_NAME}@${CI_COMMIT_SHA}"
68  when:
69    status: [success, failure]

```

4.2 Gitea Webhook 配置

在每个仓库的设置中配置:

- URL: `http://<服务器IP>:8081/hook`
- Trigger: Push events, Pull request events
- Secret: 从 Woodpecker CI 获取

五、应用服务部署配置

5.1 核心服务 Docker Compose 配置

```

1  # 创建服务部署目录
2  mkdir -p services/{postgres,influxdb,redis,kafka,eureka,config}
3
4  # PostgreSQL 配置 (共享数据库, 按 schema 隔离)
5  cat > services/docker-compose.postgres.yml <<EOF
6  version: '3.8'
7  services:
8    postgres:
9      image: postgres:15-alpine
10     container_name: smg-postgres
11     restart: unless-stopped
12     environment:
13       POSTGRES_USER: smg_admin
14       POSTGRES_PASSWORD: smg_password_2025
15     volumes:
16       - ./data/postgres:/var/lib/postgresql/data

```

```
17     - ./init-scripts:/docker-entrypoint-initdb.d
18     ports:
19     - "5432:5432"
20     networks:
21     - smg-network
22     mem_limit: 4g
23     cpus: 1.5
24
25     networks:
26     smg-network:
27     driver: bridge
28 EOF
29
30 # InfluxDB 配置（时序数据）
31 cat > services/docker-compose.influxdb.yml <<EOF
32 version: '3.8'
33 services:
34     influxdb:
35     image: influxdb:2.7-alpine
36     container_name: smg-influxdb
37     restart: unless-stopped
38     environment:
39     DOCKER_INFLUXDB_INIT_MODE: setup
40     DOCKER_INFLUXDB_INIT_USERNAME: smg_admin
41     DOCKER_INFLUXDB_INIT_PASSWORD: smg_password_2025
42     DOCKER_INFLUXDB_INIT_ORG: smartmoldguard
43     DOCKER_INFLUXDB_INIT_BUCKET: telemetry
44     volumes:
45     - ./data/influxdb:/var/lib/influxdb2
46     ports:
47     - "8086:8086"
48     networks:
49     - smg-network
50     mem_limit: 2g
51     cpus: 1
52 EOF
53
54 # Redis 配置（缓存）
55 cat > services/docker-compose.redis.yml <<EOF
56 version: '3.8'
57 services:
58     redis:
59     image: redis:7-alpine
60     container_name: smg-redis
61     restart: unless-stopped
62     command: ["redis-server", "--appendonly", "yes", "--maxmemory", "2gb", "--maxmemory-policy", "allkeys-lru"]
63     volumes:
64     - ./data/redis:/data
65     ports:
66     - "6379:6379"
67     networks:
68     - smg-network
```

```
69     mem_limit: 2.5g
70     cpus: 0.5
71 EOF
72
73 # Kafka 配置（事件总线）
74 cat > services/docker-compose.kafka.yml <<EOF
75 version: '3.8'
76 services:
77     zookeeper:
78         image: confluentinc/cp-zookeeper:7.5.0
79         container_name: smg-zookeeper
80         restart: unless-stopped
81         environment:
82             ZOOKEEPER_CLIENT_PORT: 2181
83         networks:
84             - smg-network
85         mem_limit: 512m
86         cpus: 0.3
87
88     kafka:
89         image: confluentinc/cp-kafka:7.5.0
90         container_name: smg-kafka
91         restart: unless-stopped
92         depends_on:
93             - zookeeper
94         ports:
95             - "9092:9092"
96         environment:
97             KAFKA_BROKER_ID: 1
98             KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
99             KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://<服务器IP>:9092
100             KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
101         volumes:
102             - ./data/kafka:/var/lib/kafka/data
103         networks:
104             - smg-network
105         mem_limit: 3g
106         cpus: 1
107 EOF
108
109 # Eureka 配置（服务注册与发现）
110 cat > services/docker-compose.eureka.yml <<EOF
111 version: '3.8'
112 services:
113     eureka-server:
114         image: springcloud/eureka-server:3.1.2
115         container_name: smg-eureka
116         restart: unless-stopped
117         environment:
118             - EUREKA_SERVER_PORT=8761
119             - EUREKA_CLIENT_REGISTER_WITH_EUREKA=false
120             - EUREKA_CLIENT_FETCH_REGISTRY=false
121             - EUREKA_INSTANCE_HOSTNAME=eureka-server
```

```
122     - MANAGEMENT_METRICS_EXPORT_PROMETHEUS_ENABLED=true
123     ports:
124     - "8761:8761"
125     networks:
126     - smg-network
127     mem_limit: 512m
128     cpus: 0.3
129
130 networks:
131     smg-network:
132     external: true
133 EOF
134
135 # Spring Cloud Config 配置（配置中心）
136 cat > services/docker-compose.config.yml <<EOF
137 version: '3.8'
138 services:
139     config-server:
140     image: springcloud/config-server:3.1.2
141     container_name: smg-config
142     restart: unless-stopped
143     environment:
144     - SPRING_PROFILES_ACTIVE=native
145     - SPRING_CLOUD_CONFIG_SERVER_NATIVE_SEARCH_LOCATIONS=/config
146     -
147     SPRING_CLOUD_CONFIG_SERVER_GIT_URI=https://gitea.smartmoldguard.local/smartmoldguard/c
148     onfig-repo.git
149     - SPRING_CLOUD_CONFIG_SERVER_GIT_USERNAME=admin
150     - SPRING_CLOUD_CONFIG_SERVER_GIT_PASSWORD=admin123
151     - EUREKA_CLIENT_SERVICE_URL_DEFAULTZONE=http://eureka-server:8761/eureka
152     - MANAGEMENT_METRICS_EXPORT_PROMETHEUS_ENABLED=true
153     ports:
154     - "8888:8888"
155     volumes:
156     - ./data/config:/config
157     networks:
158     - smg-network
159     depends_on:
160     - eureka-server
161     mem_limit: 512m
162     cpus: 0.3
163
164 networks:
165     smg-network:
166     external: true
167 EOF
168
169 # 启动基础服务
170 docker-compose -f services/docker-compose.postgres.yml -f services/docker-
171 compose.influxdb.yml \
172 -f services/docker-compose.redis.yml -f services/docker-compose.kafka.yml \
173 -f services/docker-compose.eureka.yml -f services/docker-compose.config.yml up -d
```

5.2 微服务 Docker Compose 模板

```
1  # 设备服务配置
2  cat > services/docker-compose.device-service.yml <<EOF
3  version: '3.8'
4  services:
5    device-service:
6      image: device-service:latest
7      container_name: smg-device-service
8      restart: unless-stopped
9      environment:
10        SPRING_PROFILES_ACTIVE: prod
11        SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/device_db
12        SPRING_DATASOURCE_USERNAME: device_user
13        SPRING_DATASOURCE_PASSWORD: device_pass_2025
14        SPRING_REDIS_HOST: redis
15        SPRING_KAFKA_BOOTSTRAP_SERVERS: kafka:9092
16        INFLUXDB_URL: http://influxdb:8086
17        EUREKA_CLIENT_SERVICE_URL_DEFAULTZONE: http://eureka-server:8761/eureka
18        SPRING_CLOUD_CONFIG_URI: http://config-server:8888
19        MANAGEMENT_METRICS_EXPORT_PROMETHEUS_ENABLED: true
20      networks:
21        - smg-network
22      depends_on:
23        - postgres
24        - redis
25        - kafka
26        - eureka-server
27        - config-server
28      mem_limit: 2g
29      cpus: 1
30      logging:
31        driver: "json-file"
32        options:
33          max-size: "10m"
34          max-file: "3"
35
36      networks:
37        smg-network:
38          external: true
39  EOF
40
41  # 控制服务配置
42  cat > services/docker-compose.control-service.yml <<EOF
43  version: '3.8'
44  services:
45    control-service:
46      image: control-service:latest
47      container_name: smg-control-service
48      restart: unless-stopped
49      environment:
50        SPRING_PROFILES_ACTIVE: prod
```

```

51     SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/control_db
52     SPRING_DATASOURCE_USERNAME: control_user
53     SPRING_DATASOURCE_PASSWORD: control_pass_2025
54     SPRING_REDIS_HOST: redis
55     SPRING_KAFKA_BOOTSTRAP_SERVERS: kafka:9092
56     EUREKA_CLIENT_SERVICE_URL_DEFAULTZONE: http://eureka-server:8761/eureka
57     SPRING_CLOUD_CONFIG_URI: http://config-server:8888
58     MANAGEMENT_METRICS_EXPORT_PROMETHEUS_ENABLED: true
59 networks:
60     - smg-network
61 depends_on:
62     - postgres
63     - redis
64     - kafka
65     - eureka-server
66     - config-server
67 mem_limit: 2g
68 cpus: 1
69 logging:
70     driver: "json-file"
71     options:
72         max-size: "10m"
73         max-file: "3"
74
75 networks:
76     smg-network:
77         external: true
78 EOF
79
80 # AI服务配置
81 cat > services/docker-compose.ai-service.yml <<EOF
82 version: '3.8'
83 services:
84     ai-service:
85         image: ai-service:latest
86         container_name: smg-ai-service
87         restart: unless-stopped
88         environment:
89             SPRING_PROFILES_ACTIVE: prod
90             SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/ai_db
91             SPRING_DATASOURCE_USERNAME: ai_user
92             SPRING_DATASOURCE_PASSWORD: ai_pass_2025
93             SPRING_REDIS_HOST: redis
94             SPRING_KAFKA_BOOTSTRAP_SERVERS: kafka:9092
95             EUREKA_CLIENT_SERVICE_URL_DEFAULTZONE: http://eureka-server:8761/eureka
96             SPRING_CLOUD_CONFIG_URI: http://config-server:8888
97             MANAGEMENT_METRICS_EXPORT_PROMETHEUS_ENABLED: true
98         networks:
99             - smg-network
100 depends_on:
101     - postgres
102     - redis
103     - kafka

```

```
104     - eureka-server
105     - config-server
106     mem_limit: 2g
107     cpus: 1
108     logging:
109       driver: "json-file"
110       options:
111         max-size: "10m"
112         max-file: "3"
113
114     networks:
115       smg-network:
116         external: true
117 EOF
118
119 # 订阅服务配置
120 cat > services/docker-compose.subscription-service.yml <<EOF
121 version: '3.8'
122 services:
123   subscription-service:
124     image: subscription-service:latest
125     container_name: smg-subscription-service
126     restart: unless-stopped
127     environment:
128       SPRING_PROFILES_ACTIVE: prod
129       SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/subscription_db
130       SPRING_DATASOURCE_USERNAME: subscription_user
131       SPRING_DATASOURCE_PASSWORD: subscription_pass_2025
132       SPRING_REDIS_HOST: redis
133       SPRING_KAFKA_BOOTSTRAP_SERVERS: kafka:9092
134       EUREKA_CLIENT_SERVICE_URL_DEFAULTZONE: http://eureka-server:8761/eureka
135       SPRING_CLOUD_CONFIG_URI: http://config-server:8888
136       MANAGEMENT_METRICS_EXPORT_PROMETHEUS_ENABLED: true
137     networks:
138       - smg-network
139     depends_on:
140       - postgres
141       - redis
142       - kafka
143       - eureka-server
144       - config-server
145     mem_limit: 2g
146     cpus: 1
147     logging:
148       driver: "json-file"
149       options:
150         max-size: "10m"
151         max-file: "3"
152
153     networks:
154       smg-network:
155         external: true
156 EOF
```

```

157
158 # 报告服务配置
159 cat > services/docker-compose.report-service.yml <<EOF
160 version: '3.8'
161 services:
162   report-service:
163     image: report-service:latest
164     container_name: smg-report-service
165     restart: unless-stopped
166     environment:
167       SPRING_PROFILES_ACTIVE: prod
168       SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/report_db
169       SPRING_DATASOURCE_USERNAME: report_user
170       SPRING_DATASOURCE_PASSWORD: report_pass_2025
171       SPRING_REDIS_HOST: redis
172       SPRING_KAFKA_BOOTSTRAP_SERVERS: kafka:9092
173       INFLUXDB_URL: http://influxdb:8086
174       EUREKA_CLIENT_SERVICE_URL_DEFAULTZONE: http://eureka-server:8761/eureka
175       SPRING_CLOUD_CONFIG_URI: http://config-server:8888
176       MANAGEMENT_METRICS_EXPORT_PROMETHEUS_ENABLED: true
177     networks:
178       - smg-network
179     depends_on:
180       - postgres
181       - redis
182       - kafka
183       - eureka-server
184       - config-server
185     mem_limit: 2g
186     cpus: 1
187     logging:
188       driver: "json-file"
189       options:
190         max-size: "10m"
191         max-file: "3"
192
193   networks:
194     smg-network:
195       external: true
196 EOF

```

5.3 API 网关配置

```

1 # API网关 Docker Compose 配置
2 cat > services/docker-compose.api-gateway.yml <<EOF
3 version: '3.8'
4 services:
5   api-gateway:
6     image: api-gateway:latest
7     container_name: smg-api-gateway
8     restart: unless-stopped
9     environment:

```

```
10     SPRING_PROFILES_ACTIVE: prod
11     EUREKA_CLIENT_SERVICE_URL_DEFAULTZONE: http://eureka-server:8761/eureka
12     SPRING_CLOUD_CONFIG_URI: http://config-server:8888
13     MANAGEMENT_METRICS_EXPORT_PROMETHEUS_ENABLED: true
14     ports:
15     - "8085:8085"
16     networks:
17     - smg-network
18     depends_on:
19     - eureka-server
20     - config-server
21     mem_limit: 1.5g
22     cpus: 0.8
23     logging:
24     driver: "json-file"
25     options:
26     max-size: "10m"
27     max-file: "3"
28
29     networks:
30     smg-network:
31     external: true
32 EOF
33
34 # API网关配置文件
35 cat > services/gateway/application.yml <<'EOF'
36 server:
37     port: 8085
38
39 spring:
40     application:
41     name: api-gateway
42     cloud:
43     gateway:
44     discovery:
45     locator:
46     enabled: true
47     lower-case-service-id: true
48     routes:
49     # 设备管理API
50     - id: device-service
51     uri: lb://device-service
52     predicates:
53     - Path=/api/v1/devices/**, /api/v1/rooms/**, /api/v1/spaces/**
54     filters:
55     - StripPrefix=2
56
57     # 环境监测API
58     - id: device-service-env
59     uri: lb://device-service
60     predicates:
61     - Path=/api/v1/environment/**
62     filters:
```

```
63         - StripPrefix=2
64
65     # 智能控制API
66     - id: control-service
67       uri: lb://control-service
68       predicates:
69         - Path=/api/v1/interventions/**, /api/v1/strategies/**, /api/v1/auto-mold-
strategy/**, /api/v1/linkage-mapping/**
70       filters:
71         - StripPrefix=2
72
73     # AI服务API
74     - id: ai-service
75       uri: lb://ai-service
76       predicates:
77         - Path=/api/v1/ai/**, /api/v1/predictions/**, /api/v1/risk-prediction/**
78       filters:
79         - StripPrefix=2
80
81     # 订阅管理API
82     - id: subscription-service
83       uri: lb://subscription-service
84       predicates:
85         - Path=/api/v1/subscription/**, /api/v1/plans/**, /api/v1/points/**
86       filters:
87         - StripPrefix=2
88
89     # 报告管理API
90     - id: report-service
91       uri: lb://report-service
92       predicates:
93         - Path=/api/v1/reports/**, /api/v1/dashboard/**
94       filters:
95         - StripPrefix=2
96
97     # B端商户管理API
98     - id: device-service-b
99       uri: lb://device-service
100      predicates:
101        - Path=/api/v1/b/**
102      filters:
103        - StripPrefix=2
104
105     # 运营端管理API
106     - id: device-service-operator
107       uri: lb://device-service
108       predicates:
109         - Path=/api/v1/operator/**
110       filters:
111         - StripPrefix=2
112
113     # 告警与工单API
114     - id: report-service-alarm
```

```
115         uri: lb://report-service
116         predicates:
117             - Path=/api/v1/alarms/**, /api/v1/workorder/**
118         filters:
119             - StripPrefix=2
120
121     management:
122         endpoints:
123             web:
124                 exposure:
125                     include: health, info, metrics, prometheus
126         metrics:
127             export:
128                 prometheus:
129                     enabled: true
130 EOF
```

六、监控与日志配置

6.1 Prometheus 服务发现配置

```
1  # 在 prometheus.yml 中配置 Eureka 服务发现
2  scrape_configs:
3      - job_name: 'eureka'
4        metrics_path: '/actuator/prometheus'
5        static_configs:
6            - targets: ['eureka-server:8761']
7
8      - job_name: 'config-server'
9        metrics_path: '/actuator/prometheus'
10       static_configs:
11           - targets: ['config-server:8888']
12
13      - job_name: 'api-gateway'
14        metrics_path: '/actuator/prometheus'
15        static_configs:
16            - targets: ['api-gateway:8085']
17
18      - job_name: 'spring-boot'
19        metrics_path: '/actuator/prometheus'
20        eureka_sd_configs:
21            - server: http://eureka-server:8761/eureka
22        relabel_configs:
23            - source_labels: [__meta_eureka_app_name]
24              target_label: application
25            - source_labels: [__meta_eureka_instance_port]
26              target_label: port
```

6.2 Grafana 仪表板配置

```
1 # 登录 Grafana (admin/admin123)
2 # 1. 添加数据源
3 #   - Prometheus: http://prometheus:9090
4 #   - Loki: http://loki:3100
5
6 # 2. 导入仪表板
7 #   - JVM Micrometer (ID: 4701)
8 #   - Spring Boot Actuator (ID: 14405)
9 #   - Loki Logs (ID: 13105)
10 #   - Eureka Dashboard (ID: 15578)
11
12 # 3. 为B端和运营端创建专用仪表板
13 #   - B端商户运营概览
14 #   - 设备状态监控
15 #   - 告警统计分析
```

6.3 日志收集配置

在每个微服务的 `logback-spring.xml` 中配置 Loki 日志收集:

```
1 <configuration>
2   <appender name="LOKI" class="com.github.loki4j.logback.Loki4jAppender">
3     <http>
4       <url>http://loki:3100/loki/api/v1/push</url>
5     </http>
6     <format>
7       <label>
8
9         <pattern>app=${spring.application.name},host=${HOSTNAME},level=%level</pattern>
10       </label>
11       <message>
12         <pattern>{} {}- ${spring.application.name} - %msg%n</pattern>
13       </message>
14       <sortByTime>true</sortByTime>
15     </format>
16   </appender>
17
18   <root level="INFO">
19     <appender-ref ref="LOKI"/>
20   </root>
21 </configuration>
```

6.4 告警规则配置

```
1 # prometheus/alert-rules.yml
2 groups:
3   - name: smartmoldguard-alerts
4     rules:
5       # 服务可用性告警
```

```

6   - alert: ServiceDown
7     expr: up == 0
8     for: 5m
9     labels:
10      severity: critical
11     annotations:
12      summary: "服务不可用"
13      description: "服务 {{ $labels.application }} 已停止运行超过5分钟"
14
15    # JVM内存使用率告警
16    - alert: HighMemoryUsage
17      expr: (jvm_memory_used_bytes / jvm_memory_max_bytes) * 100 > 85
18      for: 5m
19      labels:
20       severity: warning
21      annotations:
22       summary: "JVM内存使用率过高"
23       description: "服务 {{ $labels.application }} JVM内存使用率超过85%"
24
25    # B端商户高风险告警
26    - alert: HighRiskRooms
27      expr: sum by (application) (rate(high_risk_rooms_total[5m])) > 10
28      for: 5m
29      labels:
30       severity: critical
31      annotations:
32       summary: "高风险房间数量过多"
33       description: "B端商户高风险房间数量超过10个"
34
35    # 设备离线告警
36    - alert: DeviceOffline
37      expr: sum by (application) (rate(device_offline_events_total[5m])) > 5
38      for: 5m
39      labels:
40       severity: warning
41      annotations:
42       summary: "设备离线数量过多"
43       description: "设备离线数量超过5个"

```

七、自动化脚本与工具

7.1 一键部署脚本

```

1  #!/bin/bash
2  # /opt/smartmoldguard/deploy.sh
3
4  set -e
5
6  SERVICE_NAME=$1
7  ENVIRONMENT=$2
8

```

```

9  if [ -z "$SERVICE_NAME" ]; then
10     echo "usage: ./deploy.sh <service-name> [prod|test]"
11     exit 1
12 fi
13
14 ENVIRONMENT=${ENVIRONMENT:-test}
15 COMPOSE_FILE="services/docker-compose.$SERVICE_NAME.yml"
16
17 echo "部署服务: $SERVICE_NAME 到环境: $ENVIRONMENT"
18
19 # 1. 构建最新镜像
20 docker build -t $SERVICE_NAME:latest -f ./ $SERVICE_NAME/Dockerfile ./ $SERVICE_NAME
21
22 # 2. 执行数据库迁移（如果有）
23 docker-compose -f $COMPOSE_FILE run --rm $SERVICE_NAME \
24     java -jar app.jar --spring.profiles.active=migrate
25
26 # 3. 滚动更新服务
27 docker-compose -f $COMPOSE_FILE up -d --no-deps $SERVICE_NAME
28
29 # 4. 健康检查
30 echo "等待服务启动..."
31 sleep 30
32
33 # 获取服务端口
34 case $SERVICE_NAME in
35     "device-service") PORT=8080 ;;
36     "control-service") PORT=8081 ;;
37     "ai-service") PORT=8082 ;;
38     "subscription-service") PORT=8083 ;;
39     "report-service") PORT=8084 ;;
40     "api-gateway") PORT=8085 ;;
41     *) PORT=8080 ;;
42 esac
43
44 HEALTH=$(curl -s -o /dev/null -w "%{http_code}" http://localhost:$PORT/actuator/health)
45 if [ "$HEALTH" != "200" ]; then
46     echo "服务健康检查失败!"
47     exit 1
48 fi
49
50 echo "部署完成!"

```

7.2 一键启动所有服务脚本

```

1  #!/bin/bash
2  # /opt/smartmoldguard/start-all.sh
3
4  set -e
5
6  echo "启动所有基础服务..."

```

```

7  docker-compose -f services/docker-compose.postgres.yml -f services/docker-
  compose.influxdb.yml \
8    -f services/docker-compose.redis.yml -f services/docker-compose.kafka.yml \
9    -f services/docker-compose.eureka.yml -f services/docker-compose.config.yml up -d
10
11 # 等待基础服务启动
12 sleep 60
13
14 echo "启动微服务..."
15 docker-compose -f services/docker-compose.device-service.yml -f services/docker-
  compose.control-service.yml \
16   -f services/docker-compose.ai-service.yml -f services/docker-compose.subscription-
  service.yml \
17   -f services/docker-compose.report-service.yml -f services/docker-compose.api-
  gateway.yml up -d
18
19 echo "所有服务启动完成!"
20 echo "访问地址: http://localhost:8085/swagger-ui.html"

```

7.3 备份脚本

```

1  #!/bin/bash
2  # /opt/smartmoldguard/backup.sh
3
4  BACKUP_DATE=$(date +%Y%m%d_%H%M%S)
5  BACKUP_DIR="/opt/smartmoldguard/backups/$BACKUP_DATE"
6
7  mkdir -p $BACKUP_DIR
8
9  # 备份 PostgreSQL
10 docker exec smg-postgres pg_dumpall -U smg_admin > $BACKUP_DIR/postgres.sql
11
12 # 备份 InfluxDB
13 docker exec smg-influxdb influx backup /tmp/backup
14 docker cp smg-influxdb:/tmp/backup $BACKUP_DIR/influxdb
15
16 # 备份 Redis
17 docker exec smg-redis redis-cli BGSAVE
18 docker cp smg-redis:/data/dump.rdb $BACKUP_DIR/redis.rdb
19
20 # 备份配置文件
21 cp -r /opt/smartmoldguard/services $BACKUP_DIR/
22 cp -r /opt/smartmoldguard/data/config $BACKUP_DIR/
23
24 # 清理旧备份（保留7天）
25 find /opt/smartmoldguard/backups -type d -mtime +7 -exec rm -rf {} \;
26
27 echo "备份完成: $BACKUP_DIR"

```

7.4 B端和运营端管理工具脚本

```
1  #!/bin/bash
2  # /opt/smartmoldguard/business-tools.sh
3
4  # 功能菜单
5  show_menu() {
6      echo "=== B端和运营端管理工具 ==="
7      echo "1. 查看高风险房间列表"
8      echo "2. 导出风险报告"
9      echo "3. 批量指派保洁任务"
10     echo "4. 查看设备状态分布"
11     echo "5. 查看系统概览"
12     echo "6. 退出"
13     echo "===== "
14 }
15
16 # 查看高风险房间列表
17 view_high_risk_rooms() {
18     curl -s http://localhost:8085/api/v1/b/dashboard/high-risk-rooms | jq .
19 }
20
21 # 导出风险报告
22 export_risk_report() {
23     curl -s -o risk_report_$(date +%Y%m%d).json http://localhost:8085/api/v1/b/batch-
actions -X POST -H "Content-Type: application/json" -d '{"action": "export_report",
"roomId": []}'
24     echo "风险报告已导出: risk_report_$(date +%Y%m%d).json"
25 }
26
27 # 批量指派保洁任务
28 assign_cleaning_tasks() {
29     read -p "输入房间ID列表（逗号分隔）: " ROOM_IDS
30     read -p "输入保洁员ID: " CLEANER_ID
31     read -p "输入备注信息: " NOTE
32
33     curl -s http://localhost:8085/api/v1/b/assign-cleaning -X POST -H "Content-Type:
application/json" -d '{"roomId": [${ROOM_IDS}], "cleanerId": ${CLEANER_ID},
\note": \"${NOTE}\"}' | jq .
34 }
35
36 # 查看设备状态分布
37 view_device_status() {
38     curl -s http://localhost:8085/api/v1/operator/dashboard/device-status | jq .
39 }
40
41 # 查看系统概览
42 view_system_overview() {
43     curl -s http://localhost:8085/api/v1/operator/dashboard/overview | jq .
44 }
45
46 # 主循环
```

```

47 while true; do
48     show_menu
49     read -p "请选择操作: " OPTION
50
51     case $OPTION in
52         1) view_high_risk_rooms ;;
53         2) export_risk_report ;;
54         3) assign_cleaning_tasks ;;
55         4) view_device_status ;;
56         5) view_system_overview ;;
57         6) echo "退出工具"; break ;;
58         *) echo "无效选项, 请重新选择" ;;
59     esac
60
61     echo "按回车键继续..."
62     read
63     clear
64 done

```

7.5 非轻量级工具生产环境规划

对于生产环境, 我们推荐使用以下非轻量级工具, 提供更强大的功能和更好的扩展性:

7.5.1 GitLab CE (代码管理)

功能优势: 完整的DevOps平台, 包含代码管理、CI/CD、容器注册表、问题跟踪等

部署配置:

```

1  # 创建 GitLab CE 配置
2  cat > devops/docker-compose.gitlab.yml <<EOF
3  version: '3.8'
4  services:
5      gitlab:
6          image: gitlab/gitlab-ce:latest
7          container_name: gitlab
8          restart: unless-stopped
9          environment:
10             GITLAB_OMNIBUS_CONFIG: |
11                 external_url 'http://gitlab.smartmoldguard.local:8080'
12                 gitlab_rails['gitlab_shell_ssh_port'] = 2224
13                 postgresql['enable'] = false
14                 redis['enable'] = false
15                 gitlab_rails['db_adapter'] = 'postgresql'
16                 gitlab_rails['db_encoding'] = 'utf8'
17                 gitlab_rails['db_host'] = 'gitlab-postgres'
18                 gitlab_rails['db_port'] = '5432'
19                 gitlab_rails['db_username'] = 'gitlab'
20                 gitlab_rails['db_password'] = 'gitlab_password'
21                 gitlab_rails['redis_host'] = 'gitlab-redis'
22                 gitlab_rails['redis_port'] = '6379'
23                 gitlab_rails['gitlab_default_projects_features_issues'] = true
24                 gitlab_rails['gitlab_default_projects_features_merge_requests'] = true

```

```
25     gitlab_rails['gitlab_default_projects_features_pipelines'] = true
26     gitlab_rails['gitlab_default_projects_features_registry'] = true
27   ports:
28     - "8080:80"
29     - "8443:443"
30     - "2224:22"
31   volumes:
32     - ./data/gitlab/config:/etc/gitlab
33     - ./data/gitlab/logs:/var/log/gitlab
34     - ./data/gitlab/data:/var/opt/gitlab
35   networks:
36     - devops-net
37   depends_on:
38     - gitlab-postgres
39     - gitlab-redis
40   mem_limit: 8g
41   cpus: 2
42
43   gitlab-postgres:
44     image: postgres:15-alpine
45     container_name: gitlab-postgres
46     environment:
47       POSTGRES_DB: gitlabhq_production
48       POSTGRES_USER: gitlab
49       POSTGRES_PASSWORD: gitlab_password
50     volumes:
51       - ./data/postgres/gitlab:/var/lib/postgresql/data
52     networks:
53       - devops-net
54     restart: unless-stopped
55     mem_limit: 2g
56     cpus: 0.5
57
58   gitlab-redis:
59     image: redis:7-alpine
60     container_name: gitlab-redis
61     command: ["redis-server", "--appendonly", "yes"]
62     volumes:
63       - ./data/redis/gitlab:/data
64     networks:
65       - devops-net
66     restart: unless-stopped
67     mem_limit: 1g
68     cpus: 0.3
69
70   networks:
71     devops-net:
72       driver: bridge
73 EOF
```

7.5.2 Jenkins (CI/CD 流水线)

功能优势：成熟的CI/CD工具，拥有丰富的插件生态，支持复杂流水线设计

部署配置：

```
1 # 创建 Jenkins 配置
2 cat > devops/docker-compose.jenkins.yml <<EOF
3 version: '3.8'
4 services:
5   jenkins:
6     image: jenkins/jenkins:lts-jdk17
7     container_name: jenkins
8     restart: unless-stopped
9     user: root
10    ports:
11      - "8081:8080"
12      - "50000:50000"
13    environment:
14      - JAVA_OPTS=-Xmx4g -Xms4g
15      - JENKINS_OPTS=--prefix=/jenkins
16    volumes:
17      - ./data/jenkins:/var/jenkins_home
18      - /var/run/docker.sock:/var/run/docker.sock
19      - /usr/local/bin/docker:/usr/local/bin/docker
20    networks:
21      - devops-net
22    mem_limit: 6g
23    cpus: 2
24
25 networks:
26   devops-net:
27     external: true
28 EOF
```

7.5.3 ELK Stack (日志收集与分析)

功能优势：企业级日志管理解决方案，支持复杂日志查询、可视化和告警

部署配置：

```
1 # 创建 ELK 配置
2 cat > devops/docker-compose.elk.yml <<EOF
3 version: '3.8'
4 services:
5   elasticsearch:
6     image: docker.elastic.co/elasticsearch/elasticsearch:8.11.0
7     container_name: elasticsearch
8     environment:
9       - discovery.type=single-node
10      - xpack.security.enabled=false
11      - "ES_JAVA_OPTS=-Xms2g -Xmx2g"
12    volumes:
```

```
13     - ./data/elasticsearch:/usr/share/elasticsearch/data
14     ports:
15     - "9200:9200"
16     networks:
17     - devops-net
18     mem_limit: 4g
19     cpus: 1.5
20
21     logstash:
22     image: docker.elastic.co/logstash/logstash:8.11.0
23     container_name: logstash
24     volumes:
25     - ./devops/logstash/pipeline:/usr/share/logstash/pipeline
26     ports:
27     - "5000:5000"
28     networks:
29     - devops-net
30     depends_on:
31     - elasticsearch
32     mem_limit: 2g
33     cpus: 1
34
35     kibana:
36     image: docker.elastic.co/kibana/kibana:8.11.0
37     container_name: kibana
38     ports:
39     - "5601:5601"
40     environment:
41     - ELASTICSEARCH_HOSTS=http://elasticsearch:9200
42     networks:
43     - devops-net
44     depends_on:
45     - elasticsearch
46     mem_limit: 2g
47     cpus: 1
48
49     networks:
50     devops-net:
51     external: true
52 EOF
53
54 # 创建 Logstash 配置
55 mkdir -p devops/logstash/pipeline
56 cat > devops/logstash/pipeline/logstash.conf <<EOF
57 input {
58     tcp {
59         port => 5000
60         codec => json
61     }
62     udp {
63         port => 5000
64         codec => json
65     }
66 }
```

```

66 }
67 filter {
68     if [type] == "spring-boot" {
69         grok {
70             match => { "message" => "%{TIMESTAMP_ISO8601:timestamp}\s+%{LOGLEVEL:level}\s+%
{NUMBER:pid}\s+---\s+\[(?<thread>[A-Za-z0-9-]+)\]\s+(?<class>[A-Za-z0-9.#-]+\s+:\s+(?
<logmessage>.*)" }
71         }
72     }
73 }
74 output {
75     elasticsearch {
76         hosts => ["elasticsearch:9200"]
77         index => "smartmoldguard-%{+YYYY.MM.dd}"
78     }
79 }
80 EOF

```

7.5.4 Nexus Repository (制品库)

功能优势：企业级制品管理，支持多种格式（Maven、Docker、npm等）

部署配置：

```

1  # 创建 Nexus 配置
2  cat > devops/docker-compose.nexus.yml <<EOF
3  version: '3.8'
4  services:
5      nexus:
6          image: sonatypecommunity/nexus3:latest
7          container_name: nexus
8          restart: unless-stopped
9          ports:
10             - "8082:8081"
11             - "8083:5000" # Docker 仓库端口
12          volumes:
13             - ./data/nexus:/nexus-data
14          networks:
15             - devops-net
16          mem_limit: 4g
17          cpus: 1.5
18  EOF

```

7.5.5 SonarQube (代码质量分析)

功能优势：全面的代码质量分析，支持多种语言，提供详细的代码质量报告

部署配置：

```

1  # 创建 SonarQube 配置
2  cat > devops/docker-compose.sonarqube.yml <<EOF
3  version: '3.8'
4  services:

```

```

5  sonarqube:
6    image: sonarqube:9.9.0-community
7    container_name: sonarqube
8    restart: unless-stopped
9    environment:
10     SONAR_JDBC_URL: jdbc:postgresql://sonar-postgres:5432/sonarqube
11     SONAR_JDBC_USERNAME: sonar
12     SONAR_JDBC_PASSWORD: sonar_password
13     SONAR_ES_BOOTSTRAP_CHECKS_DISABLE: "true"
14    ports:
15     - "9000:9000"
16    volumes:
17     - ./data/sonarqube/data:/opt/sonarqube/data
18     - ./data/sonarqube/extensions:/opt/sonarqube/extensions
19     - ./data/sonarqube/logs:/opt/sonarqube/logs
20    networks:
21     - devops-net
22    depends_on:
23     - sonar-postgres
24    mem_limit: 4g
25    cpus: 1.5
26
27  sonar-postgres:
28    image: postgres:13-alpine
29    container_name: sonar-postgres
30    environment:
31     POSTGRES_USER: sonar
32     POSTGRES_PASSWORD: sonar_password
33     POSTGRES_DB: sonarqube
34     POSTGRES_HOST_AUTH_METHOD: trust
35    volumes:
36     - ./data/postgres/sonar:/var/lib/postgresql/data
37    networks:
38     - devops-net
39    restart: unless-stopped
40    mem_limit: 2g
41    cpus: 0.5
42
43  networks:
44    devops-net:
45      external: true
46  EOF

```

7.5.6 HashiCorp Vault (敏感信息管理)

功能优势：企业级密钥管理解决方案，支持动态密钥生成、加密和访问控制

部署配置：

```

1  # 创建 vault 配置
2  cat > devops/docker-compose.vault.yml <<EOF
3  version: '3.8'
4  services:

```

```

5  vault:
6    image: hashicorp/vault:1.14.0
7    container_name: vault
8    restart: unless-stopped
9    ports:
10     - "8200:8200"
11    environment:
12      VAULT_ADDR: http://0.0.0.0:8200
13      VAULT_DEV_ROOT_TOKEN_ID: "root"
14      VAULT_DEV_LISTEN_ADDRESS: "0.0.0.0:8200"
15    cap_add:
16     - IPC_LOCK
17    volumes:
18     - ./data/vault:/vault/data
19    command: server -dev
20    networks:
21     - devops-net
22    mem_limit: 2g
23    cpus: 1
24
25  networks:
26    devops-net:
27      external: true
28  EOF

```

7.5.7 非轻量级工具集成方案

CI/CD流水线集成：

```

1  // Jenkinsfile 示例，集成非轻量级工具
2  pipeline {
3    agent any
4
5    stages {
6      stage('代码质量分析') {
7        steps {
8          withSonarQubeEnv('sonarqube') {
9            sh 'mvn sonar:sonar'
10           }
11        }
12      }
13
14      stage('安全扫描') {
15        steps {
16          // 使用 Nexus IQ 或其他安全扫描工具
17          sh 'nexus-iq-cli -i ${PROJECT_ID} -s http://nexus:8081 -a
admin:admin123 -t build target/*.jar'
18        }
19      }
20
21      stage('构建 Docker 镜像') {
22        steps {
23          sh 'docker build -t ${REGISTRY}/${IMAGE_NAME}:${VERSION} .'

```

```
24         sh 'docker push ${REGISTRY}/${IMAGE_NAME}:${VERSION}'
25     }
26 }
27 }
28
29 post {
30     always {
31         junit 'target/surefire-reports/*.xml'
32         archiveArtifacts artifacts: 'target/*.jar', fingerprint: true
33     }
34     success {
35         notifySlack channel: '#builds', message: '构建成功'
36     }
37     failure {
38         notifySlack channel: '#builds', message: '构建失败'
39     }
40 }
41 }
```

日志收集集成：

```
1  <!-- logback-spring.xml 中配置 ELK 集成 -->
2  <appender name="LOGSTASH"
3  class="net.logstash.logback.appender.LogstashTcpSocketAppender">
4      <destination>logstash:5000</destination>
5      <encoder class="net.logstash.logback.encoder.LogstashEncoder">
6          <customFields>{"service_name":"device-service"}</customFields>
7      </encoder>
8  </appender>
9
10 <root level="INFO">
11     <appender-ref ref="LOGSTASH"/>
12 </root>
```

7.5.8 非轻量级工具与轻量级工具对比

工具类型	轻量级工具	非轻量级工具	优势	劣势	适用场景
代码管理	Gitea	GitLab CE	完整DevOps平台，内置CI/CD	资源消耗高	生产环境，需要完整DevOps功能
CI/CD	Woodpecker CI	Jenkins	丰富插件生态，支持复杂流水线	配置复杂，资源消耗高	生产环境，需要复杂流水线
日志管理	Loki + Promtail	ELK Stack	强大的日志查询和可视化，支持告警	资源消耗高，配置复杂	生产环境，需要高级日志分析
制品管理	Docker Hub	Nexus Repository	私有制品库，支持多种格式	资源消耗高	生产环境，需要私有制品管理

工具类型	轻量级工具	非轻量级工具	优势	劣势	适用场景
代码质量	简单测试	SonarQube	全面的代码质量分析	资源消耗高	生产环境，需要严格的代码质量控制
敏感信息管理	环境变量	HashiCorp Vault	企业级密钥管理，动态密钥生成	配置复杂	生产环境，需要严格的密钥管理

7.5.9 生产环境部署建议

1. 资源规划：
 - 建议使用独立服务器部署非轻量级工具
 - 每个工具至少分配4核8GB内存
 - 采用分布式部署，确保高可用性
2. 网络隔离：
 - 为DevOps工具创建独立网络
 - 配置防火墙，限制访问IP
 - 启用HTTPS加密传输
3. 监控告警：
 - 为DevOps工具配置监控
 - 设置告警规则，及时发现问题
 - 定期备份工具数据
4. 升级策略：
 - 制定详细的升级计划
 - 在测试环境验证升级
 - 执行升级时确保数据备份
5. 安全加固：
 - 定期更新工具版本
 - 配置强密码和访问控制
 - 启用审计日志

通过以上非轻量级工具的部署和集成，可以为SmartMoldGuard系统提供企业级的DevOps支持，确保系统在生产环境中的可靠性、安全性和可扩展性。

八、安全设计与实现

8.1 JWT认证配置

8.1.1 API网关JWT配置

```
1  # 在API网关的application.yml中添加JWT配置
2  spring:
3      security:
4          oauth2:
5              resourceserver:
6                  jwt:
7                      issuer-uri: http://auth-service:8086/auth/realms/smartmoldguard
8                      jwk-set-uri: ${spring.security.oauth2.resourceserver.jwt.issuer-
uri}/protocol/openid-connect/certs
9
10 # 添加JWT认证过滤器
11 @Configuration
12 @EnableWebFluxSecurity
13 public class SecurityConfig {
14     @Bean
15     public SecurityWebFilterChain springSecurityFilterChain(ServerHttpSecurity http) {
16         http
17             .authorizeExchange(exchanges -> exchanges
18                 .pathMatchers("/actuator/**").permitAll()
19                 .pathMatchers("/api/v1/public/**").permitAll()
20                 .anyExchange().authenticated()
21             )
22             .oauth2ResourceServer(oauth2 -> oauth2
23                 .jwt(jwt -> jwt
24                     .jwtAuthenticationConverter(jwtAuthenticationConverter())
25                 )
26             );
27         return http.build();
28     }
29
30     private JwtAuthenticationConverter jwtAuthenticationConverter() {
31         JwtAuthenticationConverter converter = new JwtAuthenticationConverter();
32         converter.setJwtGrantedAuthoritiesConverter(new
33             JwtGrantedAuthoritiesConverter());
34         return converter;
35     }
36 }
```

8.1.2 微服务JWT配置

在每个微服务的application.yml中添加JWT依赖：

```

1 <!-- Maven依赖 -->
2 <dependency>
3     <groupId>org.springframework.boot</groupId>
4     <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
5 </dependency>
6 <dependency>
7     <groupId>org.springframework.boot</groupId>
8     <artifactId>spring-boot-starter-security</artifactId>
9 </dependency>

```

8.2 HTTPS配置

8.2.1 API网关HTTPS配置

```

1 # 生成自签名证书
2 openssl req -x509 -newkey rsa:4096 -keyout key.pem -out cert.pem -days 365 -nodes
3
4 # 将证书复制到网关配置目录
5 mkdir -p services/gateway/certs
6 cp key.pem cert.pem services/gateway/certs/

```

```

1 # API网关HTTPS配置
2 server:
3     port: 8443
4     ssl:
5         enabled: true
6         key-store-type: PKCS12
7         key-store: classpath:keystore.p12
8         key-store-password: changeit
9         key-alias: smartmoldguard
10        trust-store: classpath:truststore.p12
11        trust-store-password: changeit
12
13 # Docker Compose中添加HTTPS端口映射
14 ports:
15     - "8085:8085"
16     - "8443:8443"

```

8.2.2 自动重定向HTTP到HTTPS

```

1 @Configuration
2 public class HttpsRedirectConfig {
3     @Bean
4     public webFilter httpsRedirectFilter() {
5         return (exchange, chain) -> {
6             ServerHttpRequest request = exchange.getRequest();
7             if (request.getURI().getScheme().equals("http")) {
8                 URI httpsUri = UriComponentsBuilder.fromHttpRequest(request)
9                     .scheme("https")
10                    .port(8443)

```

```

11         .build()
12         .toUri();
13         return exchange.getResponse().setComplete();
14     }
15     return chain.filter(exchange);
16 };
17 }
18 }

```

8.3 RBAC授权配置

```

1  # 在Spring Cloud Config中配置角色权限映射
2  rbac:
3    roles:
4      - name: "ROLE_ADMIN"
5        permissions:
6          - "device:read"
7          - "device:write"
8          - "control:read"
9          - "control:write"
10         - "ai:read"
11         - "ai:write"
12         - "subscription:read"
13         - "subscription:write"
14         - "report:read"
15         - "report:write"
16      - name: "ROLE_USER"
17        permissions:
18          - "device:read"
19          - "control:read"
20          - "ai:read"
21          - "subscription:read"
22          - "report:read"
23      - name: "ROLE_BUSINESS"
24        permissions:
25          - "device:read"
26          - "report:read"
27          - "b:read"
28          - "b:write"
29      - name: "ROLE_OPERATOR"
30        permissions:
31          - "device:read"
32          - "report:read"
33          - "operator:read"
34          - "operator:write"

```

8.4 敏感数据加密

8.4.1 数据库敏感数据加密

```
1  @Entity
2  public class Device {
3      @Id
4      private UUID deviceId;
5
6      @Column(length = 100)
7      private String deviceName;
8
9      @Encrypted
10     @Column(length = 255)
11     private String deviceSn;
12
13     @Encrypted
14     @Column(length = 100)
15     private String macAddress;
16
17     // 其他字段...
18 }
```

8.4.2 配置文件敏感数据加密

```
1  # 使用Spring Cloud Config加密敏感配置
2  java -jar spring-cloud-config-server-3.1.2.jar encrypt --key my-secret-key --value
   "smg_password_2025"
3  # 输出加密后的字符串: {cipher}AQB...
```

在配置文件中使用加密值:

```
1  spring:
2      datasource:
3          password: '{cipher}AQB...'
```

8.5 安全审计日志

```
1  <!-- 在logback-spring.xml中添加安全审计日志配置 -->
2  <appender name="AUDIT" class="ch.qos.logback.core.rolling.RollingFileAppender">
3      <file>${LOG_PATH}/audit.log</file>
4      <rollingPolicy class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">
5          <fileNamePattern>${LOG_PATH}/audit.%d{yyyy-MM-dd}.log</fileNamePattern>
6          <maxHistory>30</maxHistory>
7      </rollingPolicy>
8      <encoder>
9          <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{36} -
   %msg%n</pattern>
10     </encoder>
11 </appender>
```

```

12
13 <logger name="com.smartmoldguard.security" level="INFO" additivity="false">
14     <appender-ref ref="AUDIT"/>
15     <appender-ref ref="LOKI"/>
16 </logger>

```

8.6 安全加固配置

8.6.1 API网关安全加固

```

1  # 在API网关的application.yml中添加安全加固配置
2  spring:
3      cloud:
4          gateway:
5              httpclient:
6                  pool:
7                      max-idle-time: 60000
8                      max-connections: 1000
9              ssl:
10                 handshake-timeout: 10000
11                 close-notify-flush-timeout: 3000
12                 close-notify-read-timeout: 0
13             filter:
14                 request-rate-limiter:
15                 redis-rate-limiter:
16                     replenishRate: 100
17                     burstCapacity: 200
18
19  # 添加请求限流过滤器
20  @Bean
21  public RouteLocator customRouteLocator(RouteLocatorBuilder builder) {
22      return builder.routes()
23          .route("device-service", r -> r.path("/api/v1/devices/**")
24              .filters(f -> f.requestRateLimiter(c ->
25                  c.setRateLimiter(redisRateLimiter())))\
26              .uri("lb://device-service"))
27          // 其他路由...
28          .build();
29  }

```

九、与 DDD 战术设计的结合点

9.1 限界上下文的部署映射

限界上下文	微服务	部署方式	数据库 Schema
防霉预测上下文	ai-service	独立容器	ai_schema
智能控制上下文	control-service	独立容器	control_schema

限界上下文	微服务	部署方式	数据库 Schema
设备连接上下文	device-service	独立容器	device_schema
交付运维上下文	集成在 device-service	共享服务	delivery_schema
防霉报告上下文	report-service	独立容器	report_schema
客户与订阅上下文	subscription-service	独立容器	subscription_schema

8.2 聚合根的实现映射

以 Device 聚合根 为例：

```

1  // 领域模型（对应文档 4.2.1）
2  @Entity
3  @Table(name = "devices")
4  public class Device {
5      @Id
6      private UUID deviceId;
7      private String deviceSn;
8      // ... 其他属性
9
10     // 命令方法
11     public void registerDevice(String sn, String name, String location) {
12         if (this.status != DeviceStatus.UNREGISTERED) {
13             throw new DomainException("设备已注册");
14         }
15         this.deviceSn = sn;
16         this.deviceName = name;
17         this.location = location;
18         this.status = DeviceStatus.REGISTERED;
19
20         // 发布领域事件
21         DomainEventPublisher.publish(new DeviceRegisteredEvent(this));
22     }
23
24     // 其他命令方法...
25 }
26
27 // 仓储实现
28 @Repository
29 public class DeviceRepositoryImpl implements DeviceRepository {
30     @Override
31     public Device save(Device device) {
32         // 保存到 PostgreSQL
33         // 发布事件到 kafka
34         return device;
35     }
36 }

```

8.3 领域服务的实现

```
1 // InterventionService (对应文档 5)
2 @Service
3 public class InterventionService {
4     private final InterventionPlanRepository planRepository;
5     private final DeviceRepository deviceRepository;
6     private final IoTMockService iotService;
7
8     public InterventionPlan generatePlan(RiskIndex riskIndex) {
9         // 业务逻辑实现
10        InterventionPlan plan = new InterventionPlan();
11        plan.generatePlan(riskIndex);
12
13        // 保存到数据库
14        planRepository.save(plan);
15
16        // 发布事件
17        eventPublisher.publish(new InterventionGeneratedEvent(plan));
18
19        return plan;
20    }
21 }
```

8.4 模拟服务对接

```
1 // AI 模拟服务客户端 (对应文档 6.1)
2 @Component
3 public class AIMockService {
4     private final RestTemplate restTemplate;
5
6     public PredictionResult predictRisk(String deviceId, List<TelemetryData> history) {
7         String url = "http://ai-mock:8080/api/v1/ai/mock/predict";
8
9         AIMockRequest request = new AIMockRequest(deviceId, history);
10        AIMockResponse response = restTemplate.postForObject(url, request,
11        AIMockResponse.class);
12
13        return mapToPredictionResult(response);
14    }
15 }
```

九、测试策略实施

9.1 单元测试执行

```
1 # 在 Jenkins 流水线中
2 mvn test
3 # 生成报告
4 mvn surefire-report:report
```

9.2 集成测试执行

```
1 # 使用 Testcontainers
2 mvn verify -Pintegration-test
```

9.3 端到端测试

```
1 # 使用 Cypress 或 Selenium
2 npm run e2e:test
```

十、安全加固

10.1 配置防火墙

```
1 # 仅开放必要端口
2 sudo ufw allow 8080,8081,8082,8083,8084,8085,3000,9090,5601/tcp
3 sudo ufw allow 2224/tcp # GitLab SSH
4 sudo ufw enable
```

10.2 配置 HTTPS (使用自签名证书)

```
1 # 在 API 网关上配置 SSL 终止
2 # 使用 Let's Encrypt (生产环境)
3 sudo apt install certbot
4 sudo certbot certonly --standalone -d yourdomain.com
```

10.3 敏感信息管理

```
1 # 使用 Docker secrets 或环境变量
2 echo "smg_password_2025" | docker secret create db_password -
```

十一、运维手册

11.1 日常运维命令

```
1  # 查看所有服务状态
2  docker-compose -f services/ ps
3
4  # 重启某个服务
5  docker-compose -f services/docker-compose.device-service.yml restart device-service
6
7  # 查看日志
8  docker logs -f --tail 100 smg-device-service
9
10 # 性能监控
11 docker stats
12
13 # 清理无用镜像
14 docker image prune -a
```

11.2 故障排查指南

问题现象	检查步骤	解决方案
服务无法启动	1. <code>docker logs <container></code> 2. 检查端口占用	修改端口或停止冲突服务
数据库连接失败	1. <code>docker exec -it postgres psql</code> 2. 检查网络	重启数据库容器
内存不足	<code>docker stats</code>	增加 swap 或优化 JVM 参数
磁盘满	<code>du -h /opt/smartmoldguard</code>	清理旧日志和备份

十二、总结与后续优化

12.1 已完成的 DevOps 能力

- ✅ 版本控制: GitLab CE 完整代码管理
- ✅ 持续集成: Jenkins 自动化构建、测试、打包
- ✅ 持续部署: Docker Compose 滚动部署
- ✅ 制品管理: Nexus 镜像和依赖管理
- ✅ 监控告警: Prometheus + Grafana 完整监控
- ✅ 日志管理: ELK Stack 集中日志收集
- ✅ 自动化: Shell 脚本实现一键部署和备份

12.2 后续优化方向

- 1. **性能优化**: 根据监控数据调整 JVM 参数和容器资源限制
- 2. **高可用**: 研究 Docker Swarm 或 K3s 实现多节点部署
- 3. **GitOps**: 引入 ArgoCD 实现声明式部署
- 4. **混沌工程**: 使用 ChaosMesh 进行故障注入测试
- 5. **成本优化**: 评估使用轻量级替代方案 (如 Gitea 替代 GitLab)

12.3 与真实 AI/IoT 平台对接准备

当需要替换模拟服务时:

```
1  # 1. 修改服务配置
2  sed -i 's/ai-mock/ai-platform/g' services/*-service.yml
3
4  # 2. 重新部署
5  docker-compose -f services/ up -d
6
7  # 3. 验证接口
8  curl http://ai-platform/api/v1/ai/predict -X POST -d @test-data.json
```

操作规划执行时间估算:

- 环境准备: 2 小时
- DevOps 工具部署: 4 小时
- 微服务项目创建: 3 小时
- CI/CD 配置: 2 小时
- 监控日志配置: 2 小时
- **总计: 约 13 小时** (可 1-2 个工作日内完成)

所有配置文件和脚本已提供, 可直接复制使用。建议在测试环境先验证, 再应用到生产服务器。